

HOSPITAL MANAGEMENT SYSTEM

PROJECT REPORT

Submitted by

Anweshika Mehta (24BCE0790)

Drishti Priya (24BCE0798)

Prepared For

DATABASE SYSTEMS (BCSE302P) -Lab PROJECT

Submitted To

Dr. Anand Bihari

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Abstract

Modern-day hospitals may be regarded as highly complex entities that require streamlined data management and reliable storage of thousands of records. Databases utilized in the healthcare industry need to be integrated with billing systems, appointment scheduling, and pharmacy inventory management to prevent overbooking, unnecessary delays in patient discharges, billing inconsistencies, and other issues. The current paper contains a project describing the creation of a database for a Hospital Management System.

The database was built in accordance with the Third Normal Form and employs a PostgreSQL relational database management system (RDBMS). The model was constructed using multiple tables with constraints mandatory for all contributors to maintain consistency and avoid data duplication. The developed HMS database is utilized for storing patient and doctor information, as well as managing outpatient and inpatient units, pharmacy supplies, and billing procedures. The entities were related using foreign keys and combined into a relational database.

The business processes such as scheduling a doctor's shift and managing inpatient admissions and discharges cannot involve any human errors. Therefore, they were implemented using database triggers and stored procedures. For example, medical units have to be automatically discharged if they fail to settle their balances. Moreover, a special procedure was created that, upon a patient's discharge, calculates the length of stay and billing costs, including room types, doctor's fees, and medication usage.

The database can also support decision-making processes by providing decision-makers with a view that contains day rates and physician lists for wards. The views are partially indexed using B-trees to make queries simpler and faster. Overall, the project was consistent with the objectives of DBMS, including following ACID principles, implementing constraints, triggers, and transactions.

Table of Contents

	Topic	Page
1.	Introduction	4
2.	Project SCOPE	4
3.	Key Contacts and Stakeholders	5
4.	Project Resource Requirements	5
5.	ER Diagram	6-7
6.	Tables and Constraints	8-10
7.	Work break down	11
8.	REVIEW EVALUATION	12

Introduction

The Hospital Management System (HMS) is a digitizing relational database programme which has been developed in order to simplify the administrative, clinical and financial aspects of a healthcare establishment. Because of the fragmented nature of traditional hospital operations there are frequently double bookings of appointments, inaccurate tracking of inventory and delayed billing. This project involves digitizing and integrating these important processes into one unified system.

This PostgreSQL-based system is a model of an advanced Database Management System (DBMS) with its multiple features and tools. The database is relational and complies with the requirements of the third normal form (3NF) and utilizes automatic triggers and procedures to ensure referential integrity and data accuracy. The HMS (Hospital Management System) is designed to perform complex manual operations such as allocating hospital beds, adjusting the quantity of pharmaceutical items and generating combined patient billing statements automatically. It serves as a reliable backbone for the modern healthcare industry.

Project SCOPE

The project's scope comprises the essential modules for managing outpatient/inpatient lifecycles.

In-scope Modules:

Administrative Management: Organization of hospital departments and doctor profiles (specializations, consultation fees).

Patient Management: Demographic, contact information, and blood group registration.

Appointment Scheduling: Booking outpatient visits with exclusive database-level conflict resolution to prevent double-booking for the same doctor-date-time.

Inpatient & Bed Management: Ward-specific (ICU, General, Private) bed availability tracking and patient admission/discharge lifecycle management.

Pharmacy & Inventory: Medication catalog management, stock tracking, minimum stock threshold management, and inventory decrement upon prescription issuance.

Automated Billing: Generating consolidated invoices upon patient discharge with aggregated bed, attending doctor, and medication costs for a given stay.

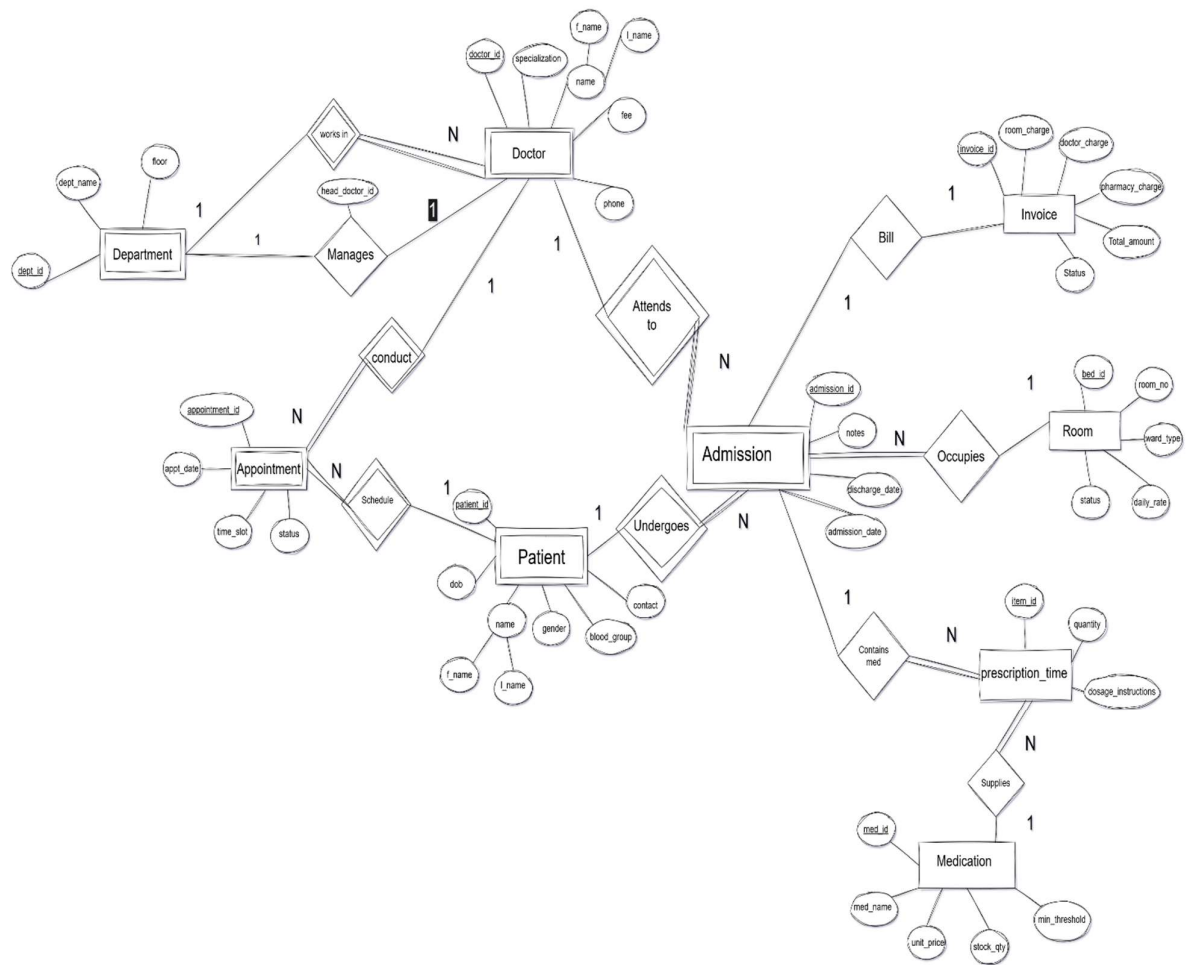
Key Contacts and Stakeholders

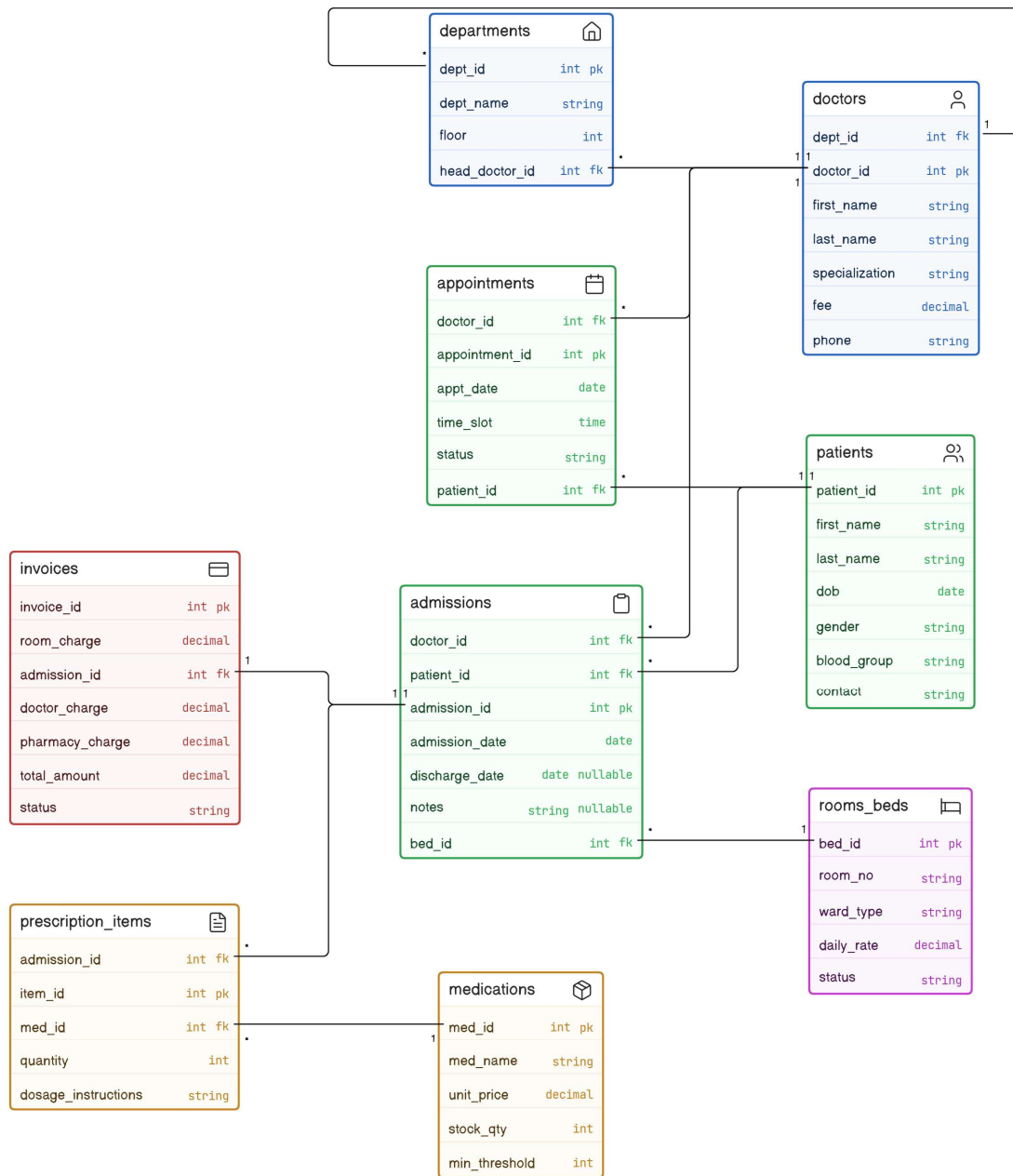
Name	Registration Number	Phone Number/Email
Anweshika Mehta	24BCE0790	8953415161
Drishti Priya	24BCE0798	9279760450

Project Resource Requirements

- ❖ PostgreSQL 16 (hosted on Supabase)
- ❖ Supabase (managed Postgres, instant REST API, and authentication)
- ❖ React 19
- ❖ TypeScript 5
- ❖ Vite 8 (build tool and development server)
- ❖ Tailwind CSS v4 (utility-first styling)
- ❖ supabase/supabase-js (client library for direct database access)
- ❖ Node.js 18+ and npm
- ❖ Visual Studio Code and a modern web browser (Chrome, Edge or Firefox)

ER Diagram





Tables and Constraints

Departments: Primary Key: dept_id | Foreign Key: head_doctor_id | Key Attributes: dept_name, floor

Doctors: Primary Key: doctor_id | Foreign Key: dept_id | Key Attributes: first_name, last_name, specialization, fee

Patients: Primary Key: patient_id | Foreign Keys: None | Key Attributes: first_name, last_name, dob, blood_group

Rooms & Beds: Primary Key: bed_id | Foreign Keys: None | Key Attributes: room_no, ward_type, daily_rate, status

Admissions: Primary Key: admission_id | Foreign Keys: patient_id, bed_id, doctor_id | Key Attributes: admission_date, discharge_date

Appointments: Primary Key: appointment_id | Foreign Keys: patient_id, doctor_id | Key Attributes: appt_date, time_slot, status

Medications: Primary Key: med_id | Foreign Keys: None | Key Attributes: med_name, unit_price, stock_qty

Prescription Items: Primary Key: item_id | Foreign Keys: admission_id, med_id | Key Attributes: quantity, dosage_instructions

Invoices: Primary Key: invoice_id | Foreign Key: admission_id | Key Attributes: room_charge, total_amount, status

Relational & Integrity Constraints

Entity Integrity (1NF & 2NF): All tables utilize atomic values and employ SERIAL PRIMARY KEYS, thereby eliminating repeating groups/partial dependencies.

Referential Integrity (3NF): Entities are appropriately normalized to separate business aspects, thereby eliminating transitive dependencies (e.g., connecting doctors and departments with dept_id).

Cascading Deletes (ON DELETE CASCADE): Deleting a dependent entity record automatically deletes child records (e.g., deleting an admission removes invoices/prescriptions).

Nullifying References (ON DELETE SET NULL): Deleting an independent entity record preserves referenced entities by NULLifying foreign keys (e.g., deleting a department retains the doctor profile but sets their dept_id to NULL).

Domain/Check Constraints: Enforces numerical logic (e.g., fee > \$0, stock_qty >= 0) and utilizes restricted vocabularies for status fields (e.g., Bed Status can only be Vacant/Occupied/Sanitizing).

Unique Constraints: A composite UNIQUE (doctor_id, appt_date, time_slot) prevents outpatient scheduling conflicts at a database level.

```
31 CREATE TABLE departments (  
32     dept_id          SERIAL PRIMARY KEY,  
33     dept_name         VARCHAR(100) NOT NULL,  
34     floor             INT NOT NULL,  
35     head_doctor_id    INT  
36 );  
37  
38 CREATE TABLE doctors (  
39     doctor_id         SERIAL PRIMARY KEY,  
40     dept_id           INT REFERENCES departments(dept_id) ON DELETE SET NULL,  
41     first_name        VARCHAR(50) NOT NULL,  
42     last_name         VARCHAR(50) NOT NULL,  
43     specialization    VARCHAR(100),  
44     fee               NUMERIC(10,2) CHECK (fee > 0),  
45     phone             VARCHAR(20)  
46 );  
47  
48 ALTER TABLE departments  
49     ADD CONSTRAINT fk_head_doctor  
50     FOREIGN KEY (head_doctor_id) REFERENCES doctors(doctor_id) ON DELETE SET NULL;  
51
```

```
56 CREATE TABLE patients (  
57     patient_id        SERIAL PRIMARY KEY,  
58     first_name        VARCHAR(50) NOT NULL,  
59     last_name         VARCHAR(50) NOT NULL,  
60     dob              DATE,  
61     gender            VARCHAR(10),  
62     blood_group       VARCHAR(5),  
63     contact           VARCHAR(20)  
64 );  
65  
66 CREATE TABLE rooms_beds (  
67     bed_id            SERIAL PRIMARY KEY,  
68     room_no           VARCHAR(10) NOT NULL,  
69     ward_type         VARCHAR(20) NOT NULL CHECK (ward_type IN ('ICU', 'General', 'Private')),  
70     daily_rate        NUMERIC(10,2) NOT NULL CHECK (daily_rate > 0),  
71     status            VARCHAR(20) NOT NULL DEFAULT 'Vacant' CHECK (status IN ('Vacant', 'Occupied', 'Sanitizing'))  
72 );  
73  
74 CREATE TABLE admissions (  
75     admission_id      SERIAL PRIMARY KEY,  
76     patient_id        INT NOT NULL REFERENCES patients(patient_id) ON DELETE CASCADE,  
77     bed_id            INT NOT NULL REFERENCES rooms_beds(bed_id),  
78     doctor_id         INT NOT NULL REFERENCES doctors(doctor_id),  
79     admission_date     TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
80     discharge_date    TIMESTAMPTZ,  
81     notes             TEXT  
82 );  
83
```

```
80 CREATE TABLE appointments (  
81     appointment_id SERIAL PRIMARY KEY,  
82     patient_id INT NOT NULL REFERENCES patients(patient_id) ON DELETE CASCADE,  
83     doctor_id INT NOT NULL REFERENCES doctors(doctor_id) ON DELETE CASCADE,  
84     appt_date DATE NOT NULL,  
85     time_slot VARCHAR(20) NOT NULL,  
86     status VARCHAR(20) NOT NULL DEFAULT 'Scheduled' CHECK (status IN ('Scheduled', 'Completed', 'Cancelled')),  
87  
88     CONSTRAINT uq_doctor_slot UNIQUE (doctor_id, appt_date, time_slot)  
89 );  
90  
91 CREATE TABLE medications (  
92     med_id SERIAL PRIMARY KEY,  
93     med_name VARCHAR(100) NOT NULL,  
94     unit_price NUMERIC(10,2) NOT NULL CHECK (unit_price > 0),  
95     stock_qty INT NOT NULL CHECK (stock_qty >= 0),  
96     min_threshold INT NOT NULL DEFAULT 15  
97 );  
98  
99 CREATE TABLE prescription_items (  
100     item_id SERIAL PRIMARY KEY,  
101     admission_id INT NOT NULL REFERENCES admissions(admission_id) ON DELETE CASCADE,  
102     med_id INT NOT NULL REFERENCES medications(med_id),  
103     quantity INT NOT NULL CHECK (quantity > 0),  
104     dosage_instructions TEXT  
105 );  
106
```

```
110  
111 CREATE TABLE invoices (  
112     invoice_id SERIAL PRIMARY KEY,  
113     admission_id INT NOT NULL REFERENCES admissions(admission_id) ON DELETE CASCADE,  
114     room_charge NUMERIC(10,2) NOT NULL DEFAULT 0,  
115     doctor_charge NUMERIC(10,2) NOT NULL DEFAULT 0,  
116     pharmacy_charge NUMERIC(10,2) NOT NULL DEFAULT 0,  
117     total_amount NUMERIC(10,2) NOT NULL DEFAULT 0,  
118     status VARCHAR(20) NOT NULL DEFAULT 'Pending' CHECK (status IN ('Pending', 'Paid')),  
119     created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
120 );  
121  
122 CREATE INDEX idx_doctors_dept ON doctors(dept_id);  
123 CREATE INDEX idx_admissions_patient ON admissions(patient_id);  
124 CREATE INDEX idx_admissions_bed ON admissions(bed_id);  
125 CREATE INDEX idx_admissions_doctor ON admissions(doctor_id);  
126 CREATE INDEX idx_admissions_active ON admissions(bed_id) WHERE discharge_date IS NULL;  
127 CREATE INDEX idx_appointments_doctor ON appointments(doctor_id, appt_date);  
128 CREATE INDEX idx_appointments_patient ON appointments(patient_id);  
129 CREATE INDEX idx_prescription_admission ON prescription_items(admission_id);  
130 CREATE INDEX idx_invoices_admission ON invoices(admission_id);  
131  
132
```

Work break down

Team Member Registration Number	Name	Work Assigned
24BCE0790	Anweshika Mehta	Database Architect & SQL Lead
24BCE0798	Drishti Priya	Frontend Developer & Integration Lead

REVIEW EVALUATION

COMPONENT	MARKS	MEMBER 1	MEMBER 2	
ER-diagram	3			
Table Creation with description and screenshot	5			
Normal form	2			
TOTAL	10 MARKS			

